Additional Node.js Best Practices — Gap Analysis and Recommendations

Supplementary document for the Node.js Best Practices report (Best_practices_in__Nodejs.docx)

> **Scope.** The company checklist in `1792819931002_image.png` (23 rows) was compared with the full text of the report. Five practices that the report already covers adequately are left out. This document lists only the practices that are missing, partially covered, or unclear.
>
> **How to read it.** *Company recommendation* is taken only from the screenshot, which gives one line per practice. *Recommended addition* and *Practical example* are supplementary technical explanations written to help you extend the report; they are not company requirements. The screenshot has no page numbers, so references use the table row (Sl No).

Summary of included practices

Sl No	Practice	Status
1	Environment Variable Management (dotenv / config)	**Missing**
3	Code Formatting with ESLint and Prettier	**Missing**
6	Process Management with PM2 or Forever	**Partially covered**
7	Secure HTTP Headers with Helmet	**Missing**
8	Modular Architecture with Separation of Concerns	**Missing**
9	Structured and Contextual Logging (Winston / Bunyan)	**Missing**
10	Middleware for Authentication, Error Handling and Logging	**Partially covered**
11	Avoiding Memory Leaks	**Partially covered**
12	Clustering for Better CPU Utilization	**Partially covered**
13	TypeScript Support (ts-node / Babel)	**Missing**
16	Handling unhandledRejection and uncaughtException	**Partially covered**
17	Environment-Specific Configuration (config package)	**Missing**
18	Secrets Management	**Missing**
19	HTTP Request Logging with morgan	**Missing**
20	Graceful Shutdown	**Partially covered**
21	Express Router Modules	**Missing**
22	Rate Limiting for Brute-Force Protection	**Missing**
23	File Length and Line Length Limits	**Needs clarification**

Part A — Missing practices (11)

Sl No 1 — Environment Variable Management (dotenv / config)

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use dotenv or config for environment variable management.

WHAT IS MISSING FROM YOUR REPORT

The report does not mention environment variables or configuration management.

RECOMMENDED ADDITION

Values that differ between environments, such as the port number, database URL and API keys, should not be hard-coded in the source files. Load them from environment variables using the `dotenv` package (which reads a `.env` file into `process.env`) or the `config` package.

Add the `.env` file to `.gitignore` so it is never committed, and keep a `.env.example` file that lists the variable names without real values. This lets the same code run in development, testing and production with different settings.

PRACTICAL EXAMPLE

```
require('dotenv').config(); // load as early as possible

const port = process.env.PORT || 3000;
const dbUrl = process.env.DATABASE_URL;
if (!dbUrl) throw new Error('DATABASE_URL is not set');
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 1 (the image shows no page number).

Sl No 3 — Code Formatting with ESLint and Prettier

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use ESLint and Prettier for consistent code formatting.

WHAT IS MISSING FROM YOUR REPORT

The report does not mention linting or automatic code formatting.

RECOMMENDED ADDITION

ESLint analyses the code and reports likely bugs and rule violations, such as unused or undefined variables. Prettier automatically formats the code so that the whole team follows one style. Used together, they keep the codebase consistent and easier to review.

Use `eslint-config-prettier` so that ESLint style rules do not conflict with Prettier, and add both tools as npm scripts so they can be run with one command.

PRACTICAL EXAMPLE

package.json scripts:

```
"scripts": {
  "lint": "eslint .",
  "format": "prettier --write ."
}
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 3 (the image shows no page number).

Sl No 7 — Secure HTTP Headers with Helmet

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Secure headers using Helmet middleware.

WHAT IS MISSING FROM YOUR REPORT

The report does not discuss HTTP security headers or Helmet.

RECOMMENDED ADDITION

Helmet is an Express middleware that sets several security-related HTTP response headers, for example Content-Security-Policy, Strict-Transport-Security and X-Content-Type-Options. These headers help protect the application against common web attacks such as cross-site scripting (XSS) and clickjacking.

Register Helmet early, before the routes, so that every response includes the headers.

PRACTICAL EXAMPLE

```
const helmet = require('helmet');
app.use(helmet());
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 7 (the image shows no page number).

Sl No 8 — Modular Architecture with Separation of Concerns

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Follow modular architecture with separation of concerns.

WHAT IS MISSING FROM YOUR REPORT

The report describes a centralized error-handling middleware, but it does not describe how the application code should be organised into modules or layers.

RECOMMENDED ADDITION

Organise the application so that each file or layer has one responsibility. Routes map URLs to controllers, controllers handle the request and response, services contain the business logic, models or repositories access the database, and middleware handles shared work such as authentication. This makes the code easier to read, test and change, and stops any single file from growing too large.

PRACTICAL EXAMPLE

Example folder layout (illustrative only; the screenshot does not prescribe a specific structure):

```
src/
  config/        settings
  routes/        URL definitions
  controllers/   request and response handling
  services/      business logic
  models/        database access
  middleware/    auth, logging, error handling
  app.js         Express app setup
  server.js      starts the server
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 8 (the image shows no page number).

Sl No 9 — Structured and Contextual Logging (Winston / Bunyan)

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Implement structured and contextual logging using Winston or Bunyan.

WHAT IS MISSING FROM YOUR REPORT

The report has no section on logging.

RECOMMENDED ADDITION

Use a logging library such as Winston or Bunyan instead of `console.log`. Write logs in a structured format (JSON) with a timestamp, a log level (error, warn, info, debug) and context fields such as request ID, user ID or route. Structured logs can be searched and filtered easily in production.

Never write passwords, tokens or other sensitive data to the logs.

PRACTICAL EXAMPLE

```
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(winston.format.timestamp(), winston.format.json()),
  transports: [new winston.transports.Console()],
});

logger.info('Order created', { orderId: 101, userId: 7, requestId: 'abc-123' });
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 9 (the image shows no page number).

Sl No 13 — TypeScript Support (ts-node / Babel)

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use ts-node or transpile with Babel for TypeScript support.

WHAT IS MISSING FROM YOUR REPORT

The report does not mention TypeScript.

RECOMMENDED ADDITION

TypeScript code is normally converted to JavaScript before it runs in Node.js. The checklist names two approaches: `ts-node`, which compiles and runs `.ts` files on the fly and is convenient during development, or transpiling with Babel (using `@babel/preset-typescript`).

Supplementary note (own suggestion, not from the company image): Babel removes types but does not check them, so run `tsc --noEmit` to catch type errors. For production, a common approach is to compile ahead of time and run the generated JavaScript.

PRACTICAL EXAMPLE

package.json scripts using ts-node and the TypeScript compiler:

```
"scripts": {
  "dev": "ts-node src/server.ts",
  "build": "tsc",
  "start": "node dist/server.js"
}
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 13 (the image shows no page number).

Sl No 17 — Environment-Specific Configuration (config package)

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Split configuration using config package per environment.

WHAT IS MISSING FROM YOUR REPORT

The report does not mention configuration files or separating settings per environment.

RECOMMENDED ADDITION

Use the `config` package to keep a separate configuration file for each environment. It loads `default.json` first and then overrides it with the file that matches `NODE_ENV` (for example `development.json` or `production.json`). Secrets should not be stored in these files; map them from environment variables using `custom-environment-variables.json`.

PRACTICAL EXAMPLE

Folder layout:

```
config/
  default.json                      shared defaults
  development.json                  development overrides
  production.json                   production overrides
  custom-environment-variables.json  maps environment variables to config keys
```

Reading a value:

```
const config = require('config');
const port = config.get('server.port'); // value depends on NODE_ENV
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 17 (the image shows no page number).

Sl No 18 — Secrets Management

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Avoid storing secrets in source code — use vaults or environment variables.

WHAT IS MISSING FROM YOUR REPORT

The report does not say how secrets (passwords, API keys, tokens) should be stored.

RECOMMENDED ADDITION

Secrets such as database passwords, API keys and JWT signing keys must never be written in the source code or committed to Git. Read them at runtime from environment variables or from a secrets vault (a managed secret store).

Supplementary note (own suggestion): if a secret is accidentally committed, treat it as exposed and replace (rotate) it, because it stays in the Git history even after the file is changed.

PRACTICAL EXAMPLE

```
// Bad: secret written in the source code
const apiKey = 'sk_live_12345';

// Good: secret read from the environment
const apiKey = process.env.PAYMENT_API_KEY;
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 18 (the image shows no page number).

Sl No 19 — HTTP Request Logging with morgan

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use morgan for HTTP request logging.

WHAT IS MISSING FROM YOUR REPORT

The report does not mention logging of incoming HTTP requests.

RECOMMENDED ADDITION

morgan is an Express middleware that logs every HTTP request, including the method, URL, status code and response time. In production, send its output to the application logger (for example the Winston logger shown under Sl No 9) so that request logs and application logs are kept in one place.

PRACTICAL EXAMPLE

```
const morgan = require('morgan');

app.use(morgan('combined', {
  stream: { write: (message) => logger.info(message.trim()) },
}));
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 19 (the image shows no page number).

Sl No 21 — Express Router Modules

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Avoid large monolithic Express apps — use Router modules.

WHAT IS MISSING FROM YOUR REPORT

The report does not discuss how to structure Express routes.

RECOMMENDED ADDITION

Do not put all routes in one large app file. Group related routes into separate Router modules (for example users, orders and products) and mount each one in the main app under a path prefix. This keeps each file small and focused, and makes routes easier to find, test and reuse.

PRACTICAL EXAMPLE

```
// routes/users.js
const router = require('express').Router();
const { listUsers, createUser } = require('../controllers/userController');

router.get('/', listUsers);
router.post('/', createUser);
module.exports = router;

// app.js
app.use('/api/users', require('./routes/users'));
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 21 (the image shows no page number).

Sl No 22 — Rate Limiting for Brute-Force Protection

MISSING

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use rate-limiter-flexible or similar for brute-force protection.

WHAT IS MISSING FROM YOUR REPORT

The report covers input validation, but it does not cover limiting how often a client can call the API.

RECOMMENDED ADDITION

Rate limiting restricts how many requests a client (for example, one IP address) can make in a given time. It slows down brute-force attacks on login endpoints and protects the API from being overloaded. The checklist names the `rate-limiter-flexible` library, or a similar one.

Supplementary note (own suggestion): `RateLimiterMemory` keeps its counts inside one process. When several servers or cluster workers are used, use a shared store such as `RateLimiterRedis`. This matches the stateless-architecture point already in your report.

PRACTICAL EXAMPLE

```
const { RateLimiterMemory } = require('rate-limiter-flexible');

// allow 5 attempts per 60 seconds for each IP address
const limiter = new RateLimiterMemory({ points: 5, duration: 60 });

app.post('/login', async (req, res, next) => {
  try {
    await limiter.consume(req.ip);
    next();
  } catch (err) {
    res.status(429).json({ error: 'Too many attempts. Try again later.' });
  }
});
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 22 (the image shows no page number).

Part B — Partially covered practices (6)

For each item, the first field shows what the report already says, so only the missing part needs to be added.

Sl No 6 — Process Management with PM2 or Forever

PARTIALLY COVERED

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use PM2 or Forever for production process management.

ALREADY IN YOUR REPORT

Engineering for Resiliency, point 7: the global error handler shuts the application down so that a process manager can restart it.

WHAT IS MISSING FROM YOUR REPORT

The report does not name a process manager or explain how one is used in production. The company checklist names PM2 or Forever.

RECOMMENDED ADDITION

In production, run the application under a process manager such as PM2 or Forever instead of starting it with plain `node`. A process manager automatically restarts the application if it crashes, which is exactly what the global error handler in the report relies on. PM2 can also restart the app after a server reboot, manage logs, and run the app in cluster mode.

PRACTICAL EXAMPLE

Basic PM2 usage:

```
npm install -g pm2
pm2 start server.js --name my-api   # start under PM2
pm2 status                          # check running apps
pm2 logs my-api                     # view logs
pm2 save                            # remember the app list for restarts
pm2 startup                         # shows the command to start PM2 on boot
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 6 (the image shows no page number).

Sl No 10 — Middleware for Authentication, Error Handling and Logging

PARTIALLY COVERED

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use middleware for authentication, error handling, and logging.

ALREADY IN YOUR REPORT

Engineering for Resiliency, point 5: a centralized error-handling middleware.

WHAT IS MISSING FROM YOUR REPORT

Authentication middleware and logging middleware are not covered, and middleware is not explained as a general mechanism.

RECOMMENDED ADDITION

Middleware functions run between the request and the route handler, so common logic is written once instead of being repeated in every route. Use middleware for three tasks: authentication (verify the user's token before protected routes), logging (record each request) and error handling (already described in your report; it is registered last).

Authentication middleware should return status 401 when the token is missing or invalid, in line with the status codes already listed in your report.

PRACTICAL EXAMPLE

Authentication middleware using JWT:

```
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Authentication required' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid token' });
  }
}

app.use('/api/orders', authenticate, ordersRouter);
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 10 (the image shows no page number).

Sl No 11 — Avoiding Memory Leaks

PARTIALLY COVERED

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Avoid memory leaks by managing resources properly.

ALREADY IN YOUR REPORT

The closing note of Engineering for Resiliency says to clean up resources when necessary, and connection pooling returns connections to the pool. Memory leaks themselves are not discussed.

WHAT IS MISSING FROM YOUR REPORT

The common causes of memory leaks in Node.js and how to prevent them.

RECOMMENDED ADDITION

A memory leak happens when memory that is no longer needed is never released, so the process slowly uses more memory until it slows down or crashes. Common causes and fixes:

- **Unbounded caches:** objects or arrays that keep growing. Use size limits, expiry times (TTL) or Redis.
- **Event listeners** added repeatedly and never removed.
- **Timers** (`setInterval`) that are never cleared.
- **Streams, files and database connections** that are not closed.
- **Global variables or closures** that hold large objects longer than needed.
- Monitor memory with `process.memoryUsage()` and heap snapshots (`node --inspect` with Chrome DevTools) to detect leaks.

PRACTICAL EXAMPLE

```
// Leak: a new listener is added on every request and never removed
app.get('/stream', (req, res) => {
  emitter.on('update', (data) => res.write(data));
});

// Fix: remove the listener when the client disconnects
app.get('/stream', (req, res) => {
  const onUpdate = (data) => res.write(data);
  emitter.on('update', onUpdate);
  req.on('close', () => emitter.off('update', onUpdate));
});
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 11 (the image shows no page number).

Sl No 12 — Clustering for Better CPU Utilization

PARTIALLY COVERED

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Use clustering for better CPU utilization.

ALREADY IN YOUR REPORT

Architectural Scaling: vertical and horizontal scaling (several servers behind a load balancer) and stateless architecture.

WHAT IS MISSING FROM YOUR REPORT

Clustering on a single machine. The report explains that Node.js handles requests on one thread, but not how to use all CPU cores.

RECOMMENDED ADDITION

A single Node.js process runs JavaScript on one CPU core. Clustering starts several worker processes, normally one per CPU core, that share the same port, so the application can use the full CPU of the machine. This can be done with Node's built-in `cluster` module or with PM2 cluster mode (`pm2 start server.js -i max`).

Workers do not share memory, so sessions and caches must be kept in a shared store such as Redis. This is the same stateless-architecture principle already in your report.

PRACTICAL EXAMPLE

```
const cluster = require('node:cluster');
const os = require('node:os');

if (cluster.isPrimary) {
  os.cpus().forEach(() => cluster.fork());          // one worker per CPU core
  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died, starting a new one`);
    cluster.fork();
  });
} else {
  require('./server'); // each worker runs the Express app
}
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 12 (the image shows no page number).

Sl No 16 — Handling unhandledRejection and uncaughtException

PARTIALLY COVERED

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Always handle unhandledRejection and uncaughtException.

ALREADY IN YOUR REPORT

Engineering for Resiliency, point 4: an unhandled promise rejection causes failures; point 7: a global error handler shuts the application down for a restart.

WHAT IS MISSING FROM YOUR REPORT

The report does not name the two Node.js process events, `unhandledRejection` and `uncaughtException`, or show how to register handlers for them.

RECOMMENDED ADDITION

Register a handler for both events. Each handler should log the error and then exit, so that the process manager can start a clean process. After an uncaught exception the application state can no longer be trusted, so the process should not keep running.

PRACTICAL EXAMPLE

Replace console.error with your application logger if one is set up:

```
process.on('unhandledRejection', (reason) => {
  console.error('Unhandled Rejection:', reason);
  process.exit(1);
});

process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  process.exit(1);
});
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 16 (the image shows no page number).

Sl No 20 — Graceful Shutdown

PARTIALLY COVERED

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

Gracefully shut down servers and close DB connections.

ALREADY IN YOUR REPORT

Engineering for Resiliency, point 7: the application is shut down so that a process manager can restart it. A clean shutdown procedure is not described.

WHAT IS MISSING FROM YOUR REPORT

Graceful shutdown steps: stop accepting new requests, let current requests finish, close database connections, then exit.

RECOMMENDED ADDITION

When the process receives SIGTERM or SIGINT (for example when PM2 restarts the app or a container is stopped), stop the HTTP server from accepting new connections, wait for requests already in progress to finish, close the database connections, and only then exit. Add a timeout so the process still exits if something hangs. This avoids dropped requests and half-finished database work.

PRACTICAL EXAMPLE

The database close call depends on the library (for example pool.end() for pg or mongoose.connection.close() for Mongoose):

```
const server = app.listen(port);

function shutdown(signal) {
  console.log(`${signal} received. Shutting down...`);
  server.close(async () => {
    await db.end();               // close the database pool
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 10000).unref(); // force exit after 10 seconds
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
```

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 20 (the image shows no page number).

Part C — Needs clarification

Sl No 23 — File Length and Line Length Limits

NEEDS CLARIFICATION

COMPANY RECOMMENDATION (FROM THE SCREENSHOT)

The row reads: "Maximum no of lines -400 to 500, Maximum characters in a line 100".

WHAT IS MISSING FROM YOUR REPORT

The report does not mention code size or line-length limits. The row cannot be interpreted fully, so no recommended addition is written for it yet.

RECOMMENDED ADDITION

Clear part: a maximum of 100 characters per line.

Unclear part: the line-count limit is written as a range ("400 to 500") and the row does not say what is counted. Please confirm with the mentor:

- Is the limit 400 lines, 500 lines, or is 400 a recommended size and 500 a hard maximum?
- Does the limit apply per file, per function, or to something else?

SOURCE REFERENCE

1792819931002_image.png, table row Sl No 23 (the image shows no page number).

Notes on the source image

- The screenshot ends at Sl No 23 and looks cropped at the bottom. If the original table has more rows, they could not be checked.
- The "Checklist (Yes/No)" column shows "Yes" for every visible row. It was not used to decide whether a practice is required; each row was treated as a company recommendation.
- All visible text was readable. No part of the image had to be guessed.